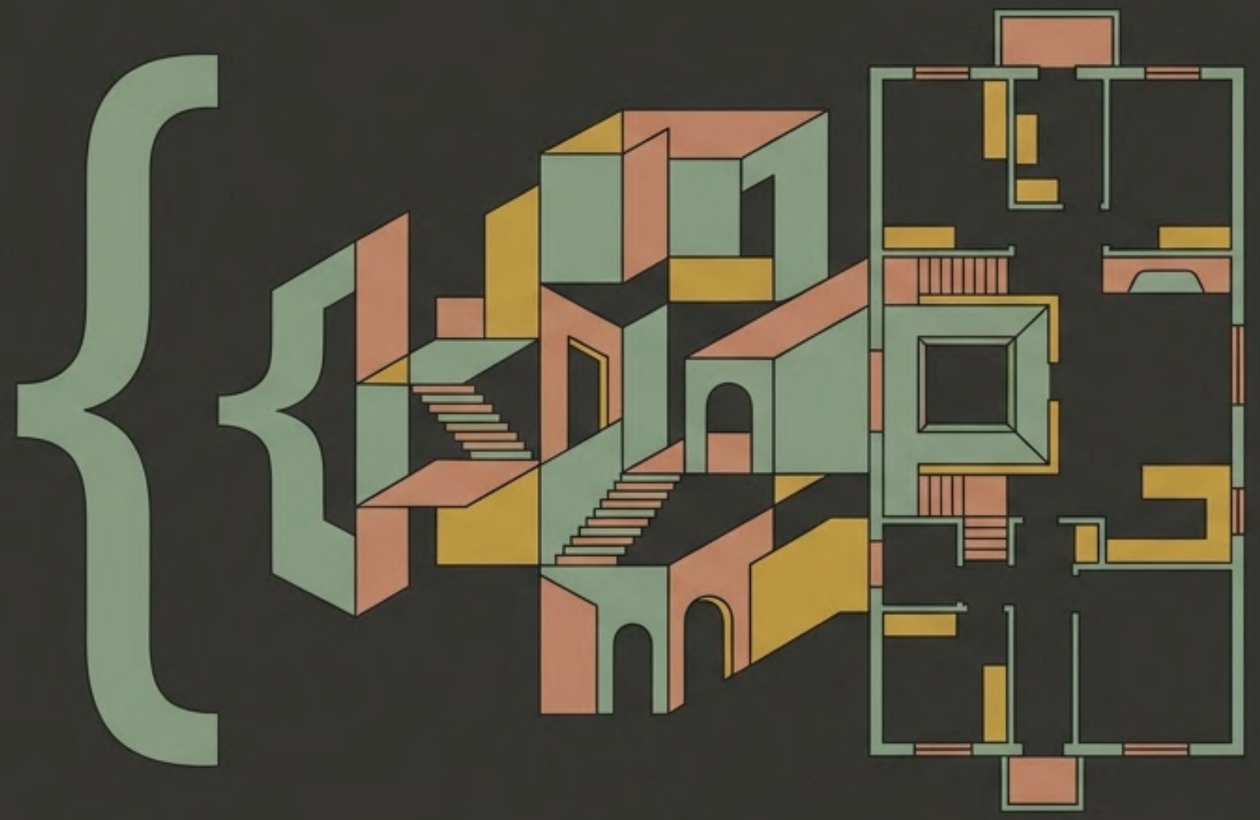


Guía de Estudio Visual

DOMINANDO LA POO EN JAVASCRIPT

De Objetos Literales a Arquitecturas Escalables

NotebookLM

La Base: El Objeto Literal

Sintaxis de Objeto Literal

```
let user = {
  name: 'John',
  age: 30,
  sayHi() {
    console.log('Hola!');
  }
};
```

Clave (String o Symbol)

Valor (Cualquier tipo de dato)

NotebookLM

Acceso y Manipulación de Datos

Notación de Punto

```
user.name = 'Pete';
```

Usado para identificadores válidos predefinidos (sin espacios, sin números al inicio).

Corchetes

```
user['likes birds'] = true;
user[varName] = 'Activo';
```

Permite strings con espacios y evaluación dinámica de variables en tiempo de ejecución.

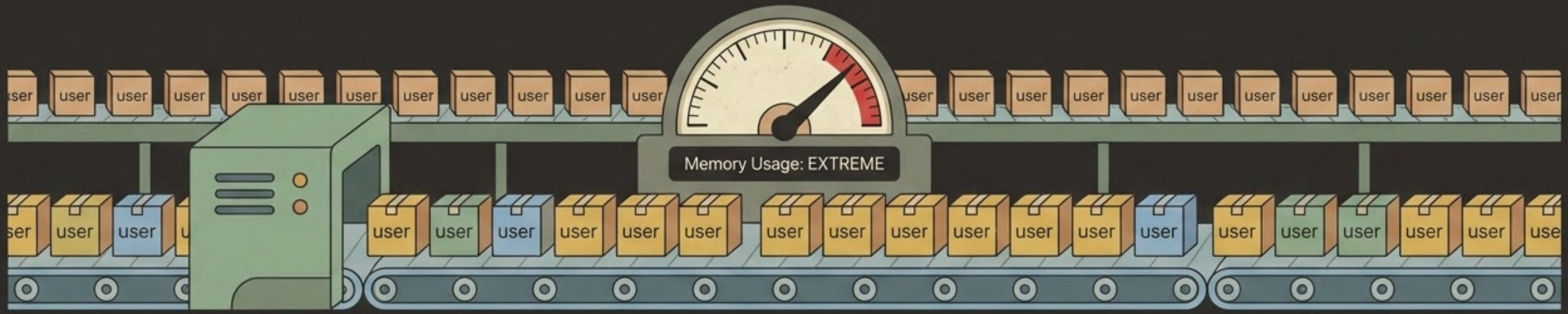
Iteración (for..in)

```
for (let key in user) {
  console.log(key, user[key]);
}
```

Recorre las claves del objeto. Las propiedades enteras se ordenan, el resto respeta el orden de creación.

NotebookLM

El Problema de la Escala




```
function createUser(name) {
  return {
    name: name,
    sayHi() { console.log('Hola'); }
  };
}
```

Redefinido 1000 veces en memoria

Ineficiencia de Memoria

- Si creamos 1,000 usuarios con una función constructora básica, el método se duplica 1,000 veces.
- **La Solución:** Necesitamos un plano arquitectónico que comparta los métodos en un solo lugar.

NotebookLM

Clases: El Plano Arquitectónico

```
class User {
  constructor(name) {
    this.name = name;
  }

  sayHi() {
    console.log(`Hola, ${this.name}`);
  }
}

let user = new User('John');
```

Inicialización: Se ejecuta automáticamente para establecer el estado.

Comportamiento compartido: Va directamente a User.prototype.

Instanciación: Crea un objeto vacío y le asigna el contexto 'this'.

Aviso: ¡No hay comas entre los métodos de la clase!

NotebookLM

¿Es solo ‘Azúcar Sintáctico’?

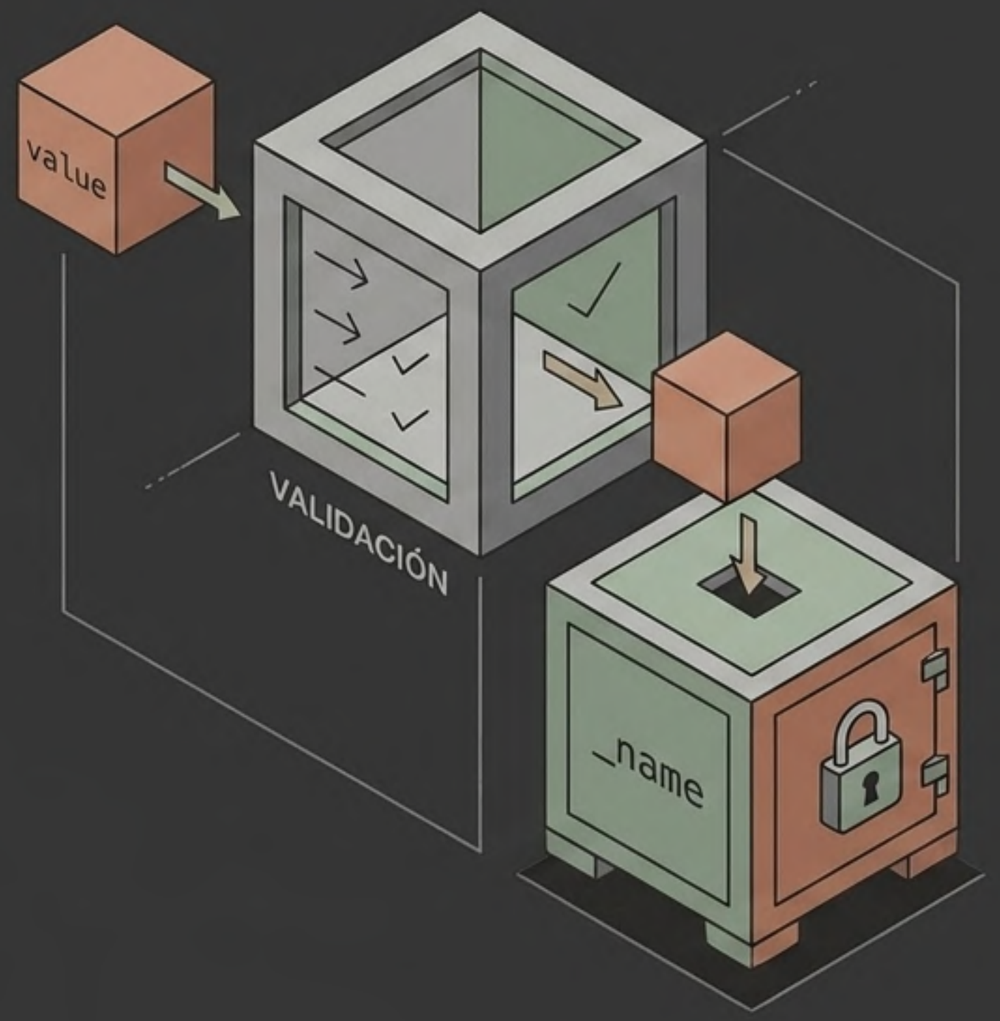
Tradicional (Función)	Moderna (Clase ES6)
`function User(name) { this.name = name; }`	`class User { constructor(name) { this.name = name; } }`

	Tradicional (Función)	Moderna (Clase ES6)
Invocación	Puede ejecutarse sin 'new' (peligroso).	Arroja error sin 'new' ([[IsClassConstructor]]).
Enumerabilidad	Métodos iterables en **for..in**.	Métodos tienen el flag **enumerable: false**.
Modo Estricto	Opcional.	Siempre aplica ‘**use strict**’ internamente.

Conclusión: No es solo sintaxis. El motor de JS trata a las clases como entidades especiales.

NotebookLM

Controlando el Acceso: Getters y Setters




```
class User {
  get name() {
    return this._name;
  }
  set name(value) {
    if (value.length < 4) {
      console.error('Nombre muy corto');
      return;
    }
    this._name = value;
  }
}
```

Concepto Clave: `_name` es una convención para indicar que la propiedad es privada. El getter/setter permite interactuar con la sintaxis de variable (`user.name = 'John'`) pero ejecutando la lógica de validación interna sin crear un bucle infinito.

NotebookLM

Campos de Clase y el Contexto de ‘this’

El Problema

```
class Button {
  click() { console.log(this.value); }
}
setTimeout(btn.click, 1000);
```

Pierde el contexto de ‘this’ (undefined)

La Solución

```
class Button {
  click = () => { console.log(this.value); }
}
setTimeout(btn.click, 1000);
```

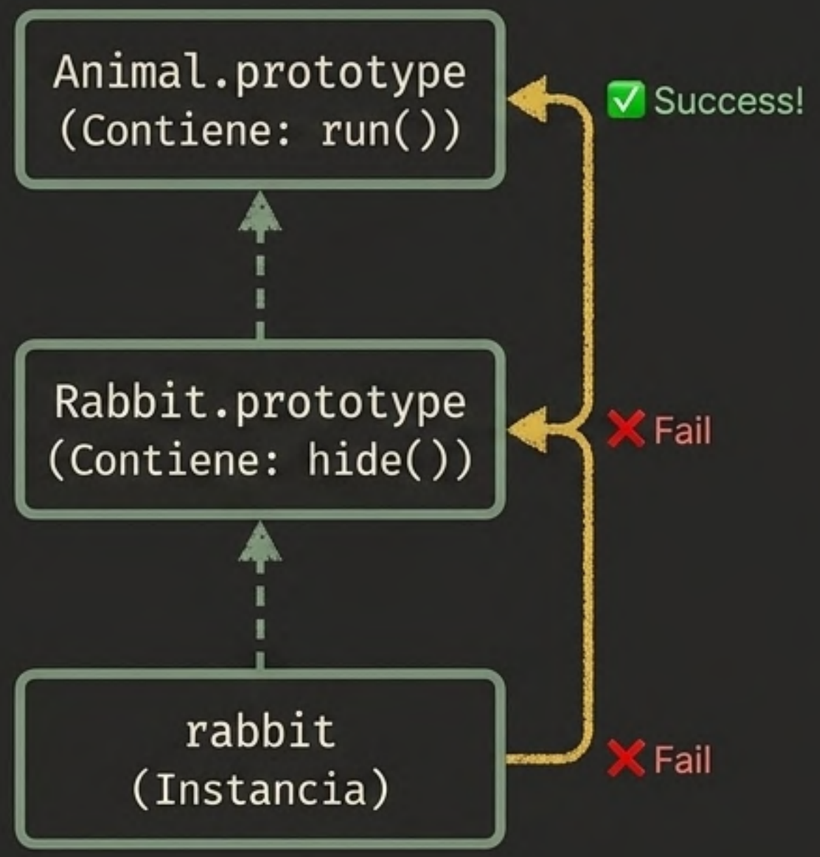
Contexto ‘this’ asegurado

Arquitectura Interna: Los métodos normales viven en el .prototype compartido. Los Campos de Clase con funciones flecha se crean individualmente por cada instancia, enlazando dinámicamente ‘this’ al objeto actual.

NotebookLM

Herencia: Extendiendo Capacidades

Cadena de Prototipos




```
class Animal {
  run() { console.log('Corriendo...'); }
}

class Rabbit extends Animal {
  hide() { console.log('Escondido!'); }
}
```

La Magia de `extends`: Esta palabra clave configura automáticamente el mecanismo interno `[[Prototype]]` de `Rabbit.prototype` para que apunte directamente a `Animal.prototype`, habilitando la delegación de métodos.

NotebookLM

La Regla de Oro: El Constructor Derivado y `super()`

```
class Rabbit extends Animal {
   constructor(name, earLength) {
      // Error: 'this' no existe aún
      super(name); // El padre crea el objeto 'this'
      this.earLength = earLength;
   }
}
```

El Motor Interno:
Los constructores derivados tienen la etiqueta [[ConstructorKind]]: 'derived'. Esperan estrictamente que el constructor padre ejecute la creación del objeto vacío asociado a 'this'. ¡Llamar a super() es un requisito absoluto antes de tocar 'this'!

NotebookLM

Sobrescritura de Métodos (Overriding)

```
class Rabbit extends Animal {

 stop() {
 super.stop(); // Llama al método del padre
 this.hide(); // Añade comportamiento propio
 }
}
```

El Motor Oculto: [[HomeObject]]

¿Cómo sabe “super” dónde buscar sin causar bucles infinitos? Los métodos en clases están permanentemente ligados a su clase original a través de la propiedad interna [[HomeObject]]. No es seguro copiar métodos que usan “super” entre distintos objetos independientes.

NotebookLM

Editorial Code Manual

La Trampa de los Campos de Clase

```
// new Rabbit() -> Imprime 'animal', no 'rabbit'!
```

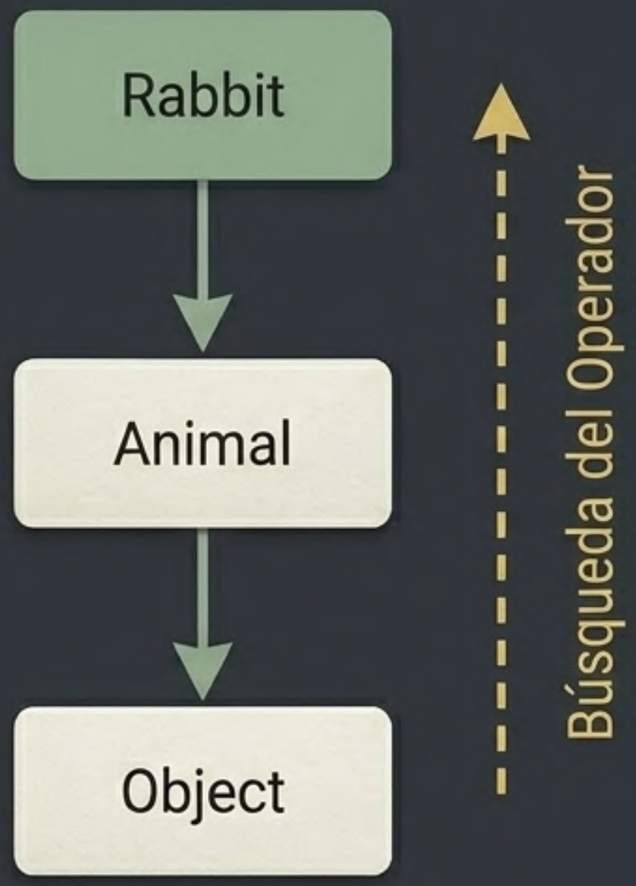
1. Ejecución de `new Rabbit()`
2. Llamada forzada a `super()` (Constructor de Animal). ↓
3. Se inicializan los campos de Animal (`name = 'animal'`).
4. Constructor de Animal hace `log` de `this.name`. Fin de super().
5. AHORA se inicializan los campos de Rabbit (`name = 'rabbit'`).

Conclusión Técnica:

En clases derivadas, los campos de clase se inicializan SIEMPRE después de super(). El padre utiliza su propio valor durante la construcción de la instancia.

NotebookLM

Verificación de Tipos con instanceof




```
let rabbit = new Rabbit();

console.log(rabbit instanceof Rabbit); // true
console.log(rabbit instanceof Animal); // true
console.log(rabbit instanceof Object); // true
```

¿Cómo funciona el algoritmo?

1. ¿Tiene la clase un método estático `Symbol.hasInstance` definido? Si es así, se ejecuta.
2. Si no, busca en la cadena: ¿Es `Class.prototype` estrictamente igual a algún prototipo en la jerarquía de `obj.__proto__`?

NotebookLM

Matriz Comparativa: Diagnóstico de Tipos

Operador / Método	Funciona Para	Retorna
`typeof`	Primitivos (Excluyendo ‘null’ ‘null’)	String (`'string'`, `'number'`)
`{}.toString.call(obj)`	Nativos, Primitivos, `Symbol.toStringTag`	String Avanzado (`'[object Array]'`)
`instanceof`	Objetos y Jerarquías de Clases POO	Booleano (`true / false`)

Usa `instanceof` para validar arquitecturas POO, y `toString` para validaciones nativas estrictas y seguras.

NotebookLM

Hoja de Trucos (Cheat Sheet) del Ecosistema

Sintaxis y Encapsulación

```
class User {
  constructor() { this._id = 1; }
  get id() { return this._id; }
}
```

Herencia y El Peaje

```
class Admin extends User {
  constructor() {
    super(); // ¡Obligatorio!
  }
}
```

Contexto Asegurado

```
class Btn {
  click = () => { /* this seguro */ }
}
```

Validación y Sobrescritura

```
super.methodName();
obj instanceof ParentClass;
```

La POO en JavaScript es una abstracción elegante sobre un motor de prototipos dinámico.

NotebookLM